

Documento de Análisis y Diseño

Proyecto 2 – Board Game Cafe

ISIS-1226: Diseño y Programación Orientado a Objetos

Universidad de los Andes

Autores

David Gabriel Arias Medina – 202512340

Edy Lane Martinez Rojas – 202510939

Laura Daniela Acosta Suárez – 202514694

Entrega única – 2026

1. Modelo de Dominio (Diagrama de Clases)

A continuación se presenta el modelo de dominio estructural del sistema, evidenciando las relaciones de herencia, composición y asociación entre las entidades. El diagrama completo se encuentra en el archivo BoardGameCafe.puml adjunto al repositorio.

1.1 Entidades y Atributos

Jerarquía de Usuarios

Usuario (Clase Abstracta): Base común de todos los actores del sistema. Define nombre, documentId, login y password. Garantiza que todas las entidades con acceso al sistema compartan credenciales.

Administrador: Extiende Usuario directamente. Tiene acceso exclusivo a funciones administrativas: gestión de inventario, aprobación de turnos y generación de informes. No puede pedir juegos prestados ni ordenar platillos.

Empleado (Clase Abstracta): Extiende Usuario. Mantiene una lista de juegosConocidos. Incluye el método puedeComprar() para validar si un empleado puede adquirir productos.

Cocinero: Empleado especializado. No requiere atributos adicionales; su tipo determina las reglas del mínimo operativo por turno.

Mesero: Empleado especializado con lista de juegosConocidos para poder enseñar los juegos catalogados como difíciles.

Cliente: Extiende Usuario. Acumula puntosFidelidad (1% de cada compra) y mantiene una lista de juegosFavoritos.

Entidades de Juego y Mesas

JuegoMesa: Representa cada juego en el inventario. Almacena id, nombre, año de publicación, empresa, rango de jugadores (min/max), categoría (Cartas, Tablero, Acción), estado, si es difícil, si es solo para adultos, y su precio de venta.

Mesa: Gestiona el estado temporal del servicio. Encapsula el clienteActual, cantidadPersonas, hayMenores, hayInfantes, alergenos, el pedidoActual de productos y los prestamosActivos.

Productos del Menú

ProductoMenu (Clase Abstracta): Define nombre, precioBase, codigoItem y descripción. Declara el método abstracto esApto(Mesa), que cada subclase implementa según sus propias restricciones.

Bebida: Especialización con atributos esAlcoholica y temperatura. El método esApto() bloquea bebidas alcohólicas en mesas con menores y bebidas calientes en mesas con juegos de categoría Acción.

Pasteleria: Especialización con lista de alergenicos. El método esApto() comprueba intersección entre los alérgenos del producto y los declarados en la mesa.

Préstamos y Ventas

Prestamo (Clase Abstracta): Define los atributos idPrestamo, juego, fechaPrestamo y fechaDevolucion. El método abstracto sonAptos(Mesa) es implementado polimórficamente por las subclases.

PrestamoCliente: Valida que la mesa tenga cliente, que no exceda 2 juegos simultáneos, que el número de personas esté en el rango del juego y que se respeten las restricciones de edad.

PrestamoEmpleado: Valida que el juego esté disponible y que el empleado no esté en turno activo.

Venta: Registra la transacción comercial. Calcula el impuesto al consumo (8% para alimentos), IVA (19% para juegos), propina y total. Aplica descuentos de fidelidad y empleado.

Gestión Operativa

Turno: Agrupa empleados por día de la semana. Implementa validarMinimoOperativo() que exige al menos 1 cocinero y 2 meseros.

Sugerencia: Contenedor de solicitudes de cambio de turno o de nuevos platillos. Tiene un flag tipoSugerencia (true = comida, false = turno) y estado estaAprobado.

Torneos

Torneo (Clase Base): Define diaSemana, juego, lista de participantes y número máximo de participantes.

TorneoAmistoso: Extiende Torneo. Añade un porcentajeDedescuento como premio (bono redimible en la siguiente compra).

TorneoCompetitivo: Extiende Torneo. Añade costo de entrada y premio en metálico.

1.2 Relaciones entre Entidades

Las relaciones clave del sistema son:

- Herencia: Usuario → Administrador, Empleado (→ Cocinero, Mesero), Cliente
- Herencia: ProductoMenu → Bebida, Pasteleria

- Herencia: Prestamo → PrestamoCliente, PrestamoEmpleado
- Herencia: Torneo → TorneoAmistoso, TorneoCompetitivo
- Asociación (Mesa – Cliente): Mesa contiene una referencia temporal al clienteActual. Esta decisión se justifica porque la ocupación es un estado efímero de la mesa, no del cliente.
- Composición (Mesa – Prestamo): Una mesa puede tener hasta 2 préstamos activos simultáneos.
- Composición (Mesa – ProductoMenu): El pedidoActual de la mesa agrupa los productos solicitados.
- Asociación (Prestamo – JuegoMesa): Cada préstamo referencia el juego solicitado.
- Asociación (Turno – Empleado): Un turno agrupa múltiples empleados; un empleado puede pertenecer a múltiples turnos.
- Asociación (Venta – Usuario): Registra quién fue el comprador (cliente o empleado).
- Asociación (SistemaBoardGameCafe): El controlador central administra todos los mapas de entidades.

2. Requerimientos Funcionales por Tipo de Usuario

2.1 Cliente

- Registrarse en el sistema con login y contraseña.
- Iniciar y cerrar sesión.
- Obtener una mesa disponible desde la cola de mesas desocupadas.
- Agregar productos del menú a su mesa (validando restricciones de edad y alérgenos).
- Solicitar préstamo de hasta 2 juegos simultáneamente (validando número de jugadores, edad, disponibilidad).
- Comprar juegos del inventario de venta, acumulando puntos de fidelidad.
- Usar puntos de fidelidad como descuento en compras futuras.
- Gestionar lista de juegos favoritos.
- Inscribirse o desinscribirse de torneos.

2.2 Empleado

- Iniciar y cerrar sesión.
- Revisar sus turnos asignados.
- Solicitar cambio de turno (sujeto a aprobación del administrador).
- Sugerir nuevos platillos al menú.
- Pedir juegos prestados siempre que no estén en turno y no haya clientes que atender.
- Comprar productos del menú y juegos con descuento del 20%.
- Inscribirse a torneos si no cubren turno ese día (torneos competitivos gratuitos para empleados, sin premio).

2.3 Administrador

- Gestionar inventario de préstamo y de venta (agregar, cambiar estado, mover entre inventarios).
- Registrar y gestionar empleados, turnos y mesas.
- Aprobar o rechazar solicitudes de cambio de turno.
- Aprobar o rechazar sugerencias de nuevos platillos.
- Crear y configurar torneos (amistosos y competitivos).
- Generar informes detallados de ventas por fecha y rubro.
- Registrar nuevos clientes y administradores.

3. Justificaciones y Proceso de Diseño

3.1 Controlador Central (Patrón Fachada)

Se introdujo la clase `SistemaBoardGameCafe` como coordinador central. Esta clase actúa como fachada entre la vista (Main/console) y las entidades del dominio. Todas las peticiones de la interfaz pasan por el controlador, que verifica el estado de sesión (`usuarioActual`) y orquesta las validaciones de negocio. Esto garantiza que las entidades de datos no asuman responsabilidades de ejecución, manteniendo una arquitectura limpia.

3.2 Eliminación de Clases Contenedoras

Las clases `Inventario`, `Historial` y `GestorMesas` del modelo conceptual inicial fueron eliminadas. Sus colecciones de datos se trasladaron como atributos `Map` y `Queue` del controlador central. Esta decisión evita el antipatrón de 'clases anémicas' y simplifica la jerarquía del sistema.

3.3 Ciclo de Vida Mesa–Cliente

La entidad `Mesa` mantiene una referencia temporal al `clienteActual`. El `Cliente` no almacena su mesa asignada. Esta decisión se basa en el ciclo de vida: un cliente es una entidad persistente (`historial`, `fidelidad`, `credenciales`), mientras que la ocupación de una mesa es un estado efímero. Mezclar ambos ciclos de vida contaminaría la entidad permanente con datos de sesión transitorios.

3.4 Delegación de Cálculos

Los cálculos matemáticos y lógicos se ejecutan dentro de las entidades que poseen los datos (`calcularTotal()` en `Venta`, `validarMinimoOperativo()` en `Turno`, `sonAptos()` en `Prestamo`, `esApto()` en `ProductoMenu`). Esto respeta el encapsulamiento y reduce el acoplamiento con el controlador.

3.5 Persistencia

Se implementó persistencia mediante serialización binaria de Java. El objeto principal `SistemaBoardGameCafe` se serializa completo (incluyendo todas sus colecciones internas) en el archivo `datos/sistema.dat`. El método `guardarSistema()` escribe el objeto mediante `ObjectOutputStream` y `cargarSistema()` lo recupera con `ObjectInputStream`. Si el archivo no existe, se crea una instancia vacía. Esta decisión fue adecuada para el

alcance del proyecto: no requiere dependencias externas, preserva todas las relaciones de objetos y mantiene el código organizado en una clase de persistencia dedicada.

4. Implementación de la Interfaz de Consola

4.1 Arquitectura de la Consola

La interfaz de usuario se implementó íntegramente en modo consola utilizando la clase Scanner de Java. Se siguió el patrón MVC de manera simplificada: la clase Main actúa como vista y controlador de flujo, delegando toda la lógica de negocio al SistemaBoardGameCafe.

La navegación se estructura en menús jerárquicos: un menú principal que distingue el tipo de usuario autenticado (administrador, empleado, cliente) y submenús funcionales para cada rol. Cada opción numérica corresponde a una funcionalidad del sistema.

4.2 Flujo de Navegación

Menú Principal (sin sesión)

1. Iniciar sesión como Administrador
2. Iniciar sesión como Empleado
3. Iniciar sesión como Cliente
4. Registrar nuevo cliente (requiere sesión de empleado/admin)
5. Salir

Menú Administrador

El menú del administrador incluye todas las funciones de gestión. Se organizó en grupos funcionales para facilitar la navegación:

- Gestión de inventario: agregar juego, cambiar estado, mover entre inventarios.
- Gestión de personal: agregar empleado, crear turno, aprobar cambios de turno.
- Gestión de mesas: agregar mesa, asignar mesa a cliente.
- Gestión de menú: agregar producto, aprobar sugerencias de empleados.
- Gestión de torneos: crear torneo amistoso, crear torneo competitivo.
- Informes: generar informe de ventas por rango de fechas.
- Persistencia: guardar estado del sistema.

Menú Empleado

- Registrar venta (mesa de cliente o compra propia).
- Procesar préstamo de juego (cliente o empleado).
- Liberar mesa al finalizar la estadía.
- Agregar plato a una mesa.

- Ver turnos asignados.
- Solicitar cambio de turno.
- Sugerir nuevo platillo.
- Inscribirse a un torneo.

Menú Cliente

- Ver juegos disponibles.
- Solicitar préstamo de juego.
- Agregar producto al pedido.
- Comprar juego del inventario de venta.
- Inscribirse o desinscribirse de torneos.
- Ver puntos de fidelidad.

4.3 Manejo de Errores en Consola

Los errores de lógica de negocio lanzados por el controlador (`SecurityException`, `IllegalArgumentException`, `IllegalStateException`) son capturados en el nivel de la consola y mostrados como mensajes descriptivos al usuario, sin interrumpir el flujo de la aplicación. El sistema regresa al menú correspondiente tras mostrar el error.

5. Suite de Pruebas (JUnit)

5.1 Descripción General

Se implementó una suite de pruebas con JUnit 5 que cubre los flujos críticos del sistema. Las pruebas verifican la seguridad de sesiones, la gestión de inventario, las operaciones de préstamo y la administración de recursos humanos.

Técnicas especiales utilizadas en las pruebas

- Reflexión Java (java.lang.reflect): Dado que el sistema tiene atributos privados sin setters públicos, las pruebas utilizan reflexión para inyectar datos directamente en los mapas internos del sistema, simulando estados preexistentes.
- Assertions.assertThrows: Para garantizar que los métodos protegidos lancen SecurityException cuando no hay sesión activa.
- Método auxiliar setAdminSession(): Establece rápidamente una sesión de administrador mediante reflexión, evitando repetición de código.

5.2 Pruebas por Categoría

Gestión de Sesiones y Seguridad

testVerificarSesionSinUsuario: Comprueba que el sistema bloquea acciones sensibles cuando usuarioActual == null, lanzando SecurityException.

testVerificarSesionConUsuario: Valida que un administrador autenticado puede ejecutar registros y operaciones administrativas sin excepción.

Lógica de Préstamos y Mesas

testProcesarPrestamo: Verifica que al prestar un juego, su estado cambie a prestado (isPrestado() == true), se incremente su contador de uso (vecesPrestado++) y se bloqueen préstamos duplicados del mismo juego.

testLimpiarMesa: Asegura que al liberar una mesa, los juegos prestados se marquen como disponibles (setPrestado(false)), se registre la fechaDevolucion y la mesa quede vacante para nuevos clientes.

testPrestamoRestriccionEdad: Verifica que un préstamo de un juego SoloAdultos sea rechazado si la mesa tiene menores.

testPrestamoRestriccionJugadores: Verifica que el préstamo sea rechazado si el número de personas en la mesa no está en el rango del juego.

Administración de Turnos

testAprobarCambioTurnoExitoso: Valida el flujo completo de aprobación de una Sugerencia de turno. Verifica la transferencia del empleado del turno origen al destino y la actualización del estado de la sugerencia.

testAprobarCambioTurnoSinPermisos: Verifica que solo usuarios con rol Administrador puedan autorizar cambios de turno, lanzando `SecurityException` para otros roles.

testValidarMinimoOperativo: Comprueba que `validarMinimoOperativo()` retorne `false` cuando el turno no tiene al menos 1 cocinero y 2 meseros.

Gestión de Inventario y Catálogo

testAgregarJuegoMesa: Valida la inserción correcta de un `JuegoMesa` en el mapa de inventario y su posterior recuperación.

testAgregarProductoMenu: Valida la inserción correcta de un `ProductoMenu` (Bebida o Pastelería) en el menú del sistema.

testAgregarJuegoVender: Valida que un juego se pueda mover al inventario de venta y que se recupere correctamente.

Operaciones de Torneo

testInscribirseTorneoEmpleado: Verifica que los empleados puedan ser añadidos a la lista de participantes de un torneo, siempre que no estén asignados al turno del día del torneo.

5.3 Estructura de una Prueba Típica

Cada prueba sigue el patrón Given-When-Then:

6. Given: Se configura el estado inicial del sistema mediante reflexión o métodos auxiliares.
7. When: Se ejecuta el método bajo prueba.
8. Then: Se verifica el estado resultante con `assertEquals`, `assertTrue`, `assertThrows`, etc.

6. Distribución Final y Correcciones

6.1 Estructura del Proyecto

La estructura de paquetes del proyecto en su entrega final es la siguiente:

Estructura de paquetes

```
boardGameCafe/
├── Main.java          (Punto de entrada y consola)
├── logic/             (Entidades del dominio)
│   ├── Usuario.java   (Clase abstracta base)
│   ├── Administrador.java
│   ├── Empleado.java  (Clase abstracta)
│   ├── Mesero.java
│   ├── Cocinero.java
│   ├── Cliente.java
│   ├── JuegoMesa.java
│   ├── Mesa.java
│   ├── ProductoMenu.java (Clase abstracta)
│   ├── Bebida.java
│   ├── Pasteleria.java
│   ├── Prestamo.java   (Clase abstracta)
│   ├── PrestamoCliente.java
│   ├── PrestamoEmpleado.java
│   ├── Venta.java
│   ├── Turno.java
│   ├── Sugerencia.java
│   ├── Torneo.java
│   ├── TorneoAmistoso.java
│   └── TorneoCompetitivo.java
├── persistencia/
│   └── Persistencia.java
├── system/
│   └── SistemaBoardGameCafe.java
└── ui/
    └── ConsolaUsuario.java (Base para consola, extensible)

datos/                (Carpeta de persistencia, fuera del código fuente)
└── sistema.dat        (Archivo binario serializado)
```

6.2 Correcciones Respecto a Entregas Anteriores

Corrección 1: Tipo de retorno de procesarPrestamo()

En la entrega anterior, el método procesarPrestamo() del controlador recibía una Mesa completa como parámetro. Se corrigió para recibir idMesa (String) y buscar la mesa internamente, manteniendo la consistencia con el resto de los métodos del controlador que operan por identificador.

Corrección 2: Manejo del historial PrestamosEmpleados

Se detectó que el historial de préstamos de empleados usaba el idJuego como clave en lugar del idPrestamo, lo que causaba sobreescritura si un mismo juego era prestado múltiples veces. Se corrigió para usar un ID único generado incrementalmente.

Corrección 3: Inicialización de torneos

El mapa torneos en SistemaBoardGameCafe no se inicializaba en el constructor, causando NullPointerException al intentar crear el primer torneo. Se añadió la inicialización correcta en el constructor de la clase.

Corrección 4: Serialización de Queue

La Queue de mesasDesocupadas utilizaba LinkedList, que implementa Serializable. Sin embargo, en una iteración anterior se había referenciado mediante la interfaz Queue sin verificar si la implementación concreta era serializable. Se aseguró el uso de LinkedList en el tipo concreto.

Corrección 5: Método esApto() en Bebida

El método esApto() de Bebida iteraba correctamente sobre los préstamos activos de la mesa para verificar juegos de categoría Acción. Se añadió la validación inversa en procesarPrestamo(): si la mesa tiene bebidas calientes en pedidoActual, no se permite el préstamo de un juego de Acción.

Corrección 6: Separación de inventarios

Se clarificó que inventario (préstamos) e inventarioVender (venta) son colecciones completamente independientes. El método moverJuegoAlInventarioVenta() remueve el juego del inventario de préstamos antes de añadirlo al de venta, evitando duplicados.

7. Programas de Demostración

Los siguientes programas de consola demuestran el correcto funcionamiento del sistema:

7.1 Programa de Persistencia

Demuestra el ciclo completo: crear datos (cliente, juego, mesa, préstamo), guardar el sistema en datos/sistema.dat, y volver a cargarlo en una nueva instancia, verificando que todos los datos se preserven correctamente.

7.2 Programa de Validación de Préstamos

Demuestra los bloqueos de seguridad del sistema:

- Rechazo si número de comensales está fuera del rango del juego.
- Rechazo si el juego es SoloAdultos y hay menores en la mesa.
- Rechazo si el juego ya está prestado.
- Rechazo de bebida caliente con juego de Acción y viceversa.
- Rechazo de pastelería con alérgenos presentes en la mesa.

7.3 Programa de Gestión de Turnos

Demuestra la validación del mínimo operativo:

- Construcción de un turno válido (2 meseros + 1 cocinero).
- Rechazo de cambio de turno que dejaría el turno con personal insuficiente.
- Aprobación exitosa de cambio cuando se cumplen las condiciones.

7.4 Programa de Ventas e Impuestos

Demuestra el cálculo correcto de impuestos y fidelidad:

- IVA del 19% sobre juegos vendidos.
- Impuesto al consumo del 8% sobre alimentos.
- Acumulación del 1% del total pagado en puntos de fidelidad.
- Descuento del 20% para empleados y 10% compartido (no acumulable).
- Uso de puntos de fidelidad como descuento en compra posterior.

7.5 Programa de Control de Administrador

Demuestra las potestades exclusivas del administrador:

- Cambio de estado de un juego (nuevo → falta una pieza → malo → desaparecido).
- Movimiento de juego del inventario de préstamos al de venta.
- Creación de un torneo competitivo y validación de copias disponibles.
- Generación del informe de ventas por rango de fechas.

8. Declaración sobre el Uso de Herramientas de Inteligencia Artificial

Para la elaboración de la presente entrega se utilizaron herramientas de inteligencia artificial generativa de manera ética, responsable y como apoyo complementario al trabajo del equipo. A continuación se detalla el alcance y la naturaleza de dicho uso:

8.1 Usos Permitidos y Realizados

a) Optimización de implementaciones ya funcionales

Una vez que el equipo implementó la lógica correcta de un método, se consultó a la IA para identificar versiones más eficientes o idiomáticas del mismo código. Por ejemplo:

"Implementé este método de la siguiente manera [...]. Ya que funciona correctamente, ¿podrías ayudarme a encontrar una versión más eficiente o legible?"

En estos casos, la lógica de negocio, los llamados a otros métodos y la toma de decisiones de diseño fueron obra íntegra del equipo. La IA únicamente sugirió refactorizaciones estilísticas o estructurales sobre código ya correcto.

b) Generación de bases para entrada de datos por consola

Dado el alto volumen de entradas requeridas por Scanner (una por cada campo de cada entidad), se utilizó la IA para generar código base con prompts del tipo:

"Crea una base para capturar los datos necesarios para usar la función procesarPrestamo(...)"

En todos estos casos, la lógica de validación, los llamados al controlador SistemaBoardGameCafe y el manejo de errores fueron implementados y verificados por el equipo. La IA generó únicamente la estructura de lectura de Scanner, no la lógica de negocio asociada.

c) Redacción del informe

El documento de diseño fue redactado inicialmente por el equipo como borrador con las ideas, decisiones y justificaciones propias. Posteriormente, se usó la IA para mejorar la redacción, ortografía y estilo técnico, manteniendo fielmente el contenido original del equipo. La estructura, las decisiones de diseño y el análisis son producto exclusivo del trabajo del equipo.

8.2 Usos No Realizados

Se deja expresa constancia de que la inteligencia artificial NO fue utilizada para:

- Generar la lógica de negocio del sistema.
- Diseñar la jerarquía de clases ni las relaciones entre entidades.
- Definir las reglas de validación de préstamos, ventas, turnos ni torneos.
- Implementar los algoritmos de cálculo de impuestos, fidelidad ni descuentos.
- Diseñar o implementar la suite de pruebas JUnit.
- Tomar decisiones arquitectónicas sobre el patrón de diseño, la persistencia ni la distribución de responsabilidades.

8.3 Responsabilidad del Equipo

La toma de decisiones arquitectónicas, el proceso esencial de diseño orientado a objetos (incluyendo la definición de clases, métodos y responsabilidades), la construcción lógica del código fuente y la validación del correcto funcionamiento del sistema fueron realizados y analizados íntegramente por los autores del proyecto: David Gabriel Arias Medina, Edy Lane Martinez Rojas y Laura Daniela Acosta Suárez.

Las herramientas de inteligencia artificial fueron utilizadas como apoyo metodológico complementario, de manera análoga al uso de documentación técnica, foros especializados o tutoriales. Ninguna parte de la lógica central del sistema fue delegada a la inteligencia artificial.

David G. Arias Medina
202512340

Edy L. Martinez Rojas
202510939

Laura D. Acosta Suárez
202514694